

# Week 4 AI Learning Plan

Production-Ready RAG — 5 days Mon-Fri — ~2 hrs/day — LangSmith + Streamlit polish + Team demo

NLP layer

LLM layer

GenAI layer

YouTube

GFG

Free Course

Week 4 goal

A polished, debugged Streamlit RAG app running on real org documents — with LangSmith tracing, error handling, guardrails, and session metrics. Presented to your team. This is your 4-week project deliverable.

WEEK 4 OVERVIEW

■ click any row to jump to that day

|       |                                                                 |          |
|-------|-----------------------------------------------------------------|----------|
| Day 1 | RAG quality tuning   Fix weak answers systematically            | ~2 hrs   |
| Day 2 | LangSmith debugging   See inside every LLM call                 | ~2 hrs   |
| Day 3 | Error handling + guardrails   Make the app reliable             | ~1.5 hrs |
| Day 4 | Streamlit polish   From working to impressive                   | ~1.5 hrs |
| Day 5 | Team demo + project presentation   4 weeks of work in 5 minutes | ~2 hrs   |

Install everything once before Day 1

```
pip install langsmith langchain langchain-google-genai
pip install langchain-community chromadb pypdf streamlit
```

Day 1

RAG quality tuning | Fix weak answers systematically

~2 hrs

ORG CONNECTION:

Documenting your tuning results shows your team you understand the system — not just that it works

NLP layer

Theory — Why RAG gives wrong answers and how to fix each cause

TOPICS IN THIS SECTION:

chunk\_size

chunk\_overlap

k\_parameter

RAG evaluation

RAGAS

Prompt tuning

TOPICS TO COMPLETE — check each as you understand it

☐ Three root causes of RAG failure: retrieval, chunking, and prompt

☐ How chunk\_size affects precision vs context tradeoff

☐ How k (top-k) affects recall vs noise tradeoff

☐ How to test RAG quality systematically with known-answer questions

☐ What RAGAS is and what metrics it measures (faithfulness, relevance)

RAG failures have three root causes: wrong chunks retrieved (retrieval problem), right chunks with irrelevant content (chunk size problem), or right content passed poorly to LLM (prompt problem). Each has a different fix. chunk\_size controls how much text per chunk — smaller is more precise but loses context. k controls how many chunks to retrieve — more increases recall but adds noise. The system prompt controls what the LLM does with the retrieved content.

FOCUS ON

Test with known-answer questions first | Tune one parameter at a time | chunk\_size=500 and k=4 are good defaults — only change if you see failures

SKIP FOR NOW

RAGAS full setup today — just understand what it measures

RESOURCES

☐

YouTube

Production RAG with LangChain full course (May 2026)

☐

YouTube

Production RAG hybrid+reranking+HyDE (Aug 2025)

☐

Blog

RAG with LangChain — GFG quality section

☐

Blog

RAGAS — evaluate your RAG quality

READ MORE — TOPIC REFERENCES

GeeksForGeeks

RAG with LangChain — tuning and quality

GeeksForGeeks

RAGAS — RAG evaluation framework

GeeksForGeeks

Retrievals in LangChain — k parameter

— Take a 10 min break before starting practice —

GenAI layer

Practice — Tune chunk\_size, k, and prompt on your org document

TOPICS IN THIS SECTION:

chunk\_size=200

chunk\_size=500

chunk\_size=1000

k=2 vs k=6

answer quality

TOPICS TO COMPLETE — check each as you finish the task

☐

Pick 5 questions with known answers from your org document

☐

Run baseline: chunk\_size=500, k=4, note which answers are wrong

☐

Try chunk\_size=200 — does precision improve for specific questions?

☐

Try chunk\_size=1000 — does context improve for broad questions?

☐

Try k=2 vs k=6 — compare quality and token usage

☐

Rewrite system prompt to be more explicit about format

☐

Document best settings in a comment block at top of notebook

WATCH BEFORE CODING

YouTube

CONCEPTS

Production RAG with LangChain full course (May 2026)

READ MORE — TOPIC REFERENCES (verified links)

GFG

GFG — RAG with LangChain (tuning section)

GFG

GFG — RAGAS evaluation framework

COMMON ERRORS + QUICK FIXES

All chunk sizes give same result

→ Your questions might be too easy — test with specific technica

k=8 gives worse answers than k=4

→ Too many chunks = noise — more is not always better, 4-6 is us

Re-embedding takes too long

→ Normal for large PDFs — add time.sleep(1) between batch embedd

Expected keyword check fails on correct answer

→ Use a partial match or just read answers manually — automated

Step 1 Set up quality test baseline

New notebook. Load your Week 3 org document and test with 5 known-answer questions.

```

from langchain_google_genai import ChatGoogleGenerativeAI, GoogleGenerativeAIEmbeddings
from langchain_community.vectorstores import Chroma
from langchain.chains import create_retrieval_chain
from langchain.chains.combine_documents import create_stuff_documents_chain
from langchain_core.prompts import ChatPromptTemplate

# 5 questions where you already know the correct answer
TEST_QUESTIONS = [
    {"q": "your question 1", "expected": "expected answer 1"},
    {"q": "your question 2", "expected": "expected answer 2"},
    # add 5 total
]

def build_rag(chunk_size, k):
    """Build RAG chain with given parameters."""
    from langchain_community.document_loaders import PyPDFLoader
    from langchain.text_splitter import RecursiveCharacterTextSplitter
    pages = PyPDFLoader("your_org_doc.pdf").load()
    chunks = RecursiveCharacterTextSplitter(
        chunk_size=chunk_size, chunk_overlap=int(chunk_size*0.1)
    ).split_documents(pages)
    embeddings = GoogleGenerativeAIEmbeddings(
        model="models/gemini-embedding-001", google_api_key="your-key")
    vs = Chroma.from_documents(chunks, embeddings)
    llm = ChatGoogleGenerativeAI(model="gemini-2.0-flash", google_api_key="your-key")
    prompt = ChatPromptTemplate.from_messages([
        ("system", "Answer ONLY from context. Cite page numbers.\n{context}"),
        ("human", "{input}")
    ])
    return create_retrieval_chain(vs.as_retriever(search_kwargs={"k":k}),
                                create_stuff_documents_chain(llm, prompt))

```

**Note:** This is the same RAG chain you built in Week 3. You are now systematically testing it instead of using it ad-hoc.

## Step 2 Compare chunk sizes

New cell. Run the same 5 questions with 3 different chunk sizes.

```

def test_rag(rag_chain, label):
    correct = 0
    for item in TEST_QUESTIONS:
        result = rag_chain.invoke({"input": item["q"]})
        answer = result["answer"]
        # Simple check: does expected keyword appear in answer?
        if item["expected"].lower() in answer.lower():
            correct += 1
    print(f"{label}: {correct}/{len(TEST_QUESTIONS)} correct")
    return correct

for chunk_size in [200, 500, 1000]:
    chain = build_rag(chunk_size=chunk_size, k=4)
    test_rag(chain, f"chunk_size={chunk_size}, k=4")

```

**Week 3 connection:** The best `chunk_size` for your document depends on how your content is structured. Policy docs with short bullet points → smaller chunks. Long narrative documents → larger chunks.

### Step 3 Find best k and update your Streamlit app

New cell. Test k values then update your Week 3 app with the best settings.

```
best_chunk = 500 # replace with your best from above

for k in [2, 4, 6, 8]:
    chain = build_rag(chunk_size=best_chunk, k=k)
    test_rag(chain, f"chunk_size={best_chunk}, k={k}")

# Best settings comment block — add this to your Streamlit app
best_settings = """
# TUNED RAG SETTINGS (Week 4 Day 1 tuning results)
# chunk_size = <your best>
# k = <your best>
# These were tested on: <your document name>
"""
```

**Note:** Write the best settings in a comment block at the top of your Streamlit app. Future you will thank present you.

#### End of Day 1 — mark each one when done

- ☐ 5 known-answer test questions prepared
- ☐ Tested 3 chunk sizes and noted which gave best results
- ☐ Tested k=2, 4, 6 and noted the optimal value
- ☐ Best settings documented in a comment block
- ☐ Streamlit app updated with best chunk\_size and k
- ☐ Notebook saved as week4\_day1\_rag\_tuning.ipynb

**NEXT:** Day 2: Connect LangSmith to your RAG chain — two environment variables and every LLM call gets traced. You will see exactly what prompt was sent, what came back, and how many tokens each step used.

MY NOTES — write your key takeaway before moving on

Takeaway: \_\_\_\_\_  
 Surprised by: \_\_\_\_\_  
 Question: \_\_\_\_\_

## Day 2 LangSmith debugging | See inside every LLM call

~2 hrs

**ORG CONNECTION:** LangSmith is what your org will require before deploying any AI app to production — you now know how to use it

LLM layer **Theory — LangSmith tracing — what it captures and why it matters**

#### TOPICS IN THIS SECTION:

LangSmith

Observability

Latency per step

Tracing

Token usage

Production debug

TOPICS TO COMPLETE — check each as you understand it

☐

What LangSmith traces: prompt, response, tokens, latency per step

☐

How two environment variables enable tracing with no code changes

☐

What a LangSmith trace looks like in the dashboard

☐

How to read a trace to find which step caused a wrong answer

☐

LANGCHAIN\_TRACING\_V2 and LANGCHAIN\_API\_KEY setup

LangSmith is LangChain's observability platform. It automatically traces every LLM call in your chain and logs: the full prompt sent, the model response, token counts, latency per step, and any errors. In production, when a user reports a wrong answer, LangSmith shows you exactly which retrieved chunks caused it. You set it up with two environment variables — no code changes needed.

FOCUS ON

Two env variables = full tracing, no code changes | Trace shows full prompt sent to LLM — this is the most useful debugging tool

SKIP FOR NOW

LangSmith evaluators, datasets, annotation queues — after production deployment

RESOURCES

☐

YouTube

LangSmith + LangChain Full AI Agent Tracing and Debugging (Feb 2026)

☐

YouTube

Trace and Debug LangChain Agents with LangSmith (Apr 2026)

☐

YouTube

LangSmith Tracing Tutorial step by step (Jan 2026)

☐

Blog

Debugging and Testing LLMs in LangSmith — GFG

☐

Blog

Tracing with LangChain — GFG

READ MORE — TOPIC REFERENCES

GeeksForGeeks

Debugging and Testing LLMs in LangSmith

GeeksForGeeks

Tracing with LangChain — full setup guide

GeeksForGeeks

Introduction to LangSmith

GeeksForGeeks

Complete LangChain Ecosystem — LangSmith section

— Take a 10 min break before starting practice —

GenAI layer

Practice — Connect LangSmith to your RAG chain and read traces

TOPICS IN THIS SECTION:

LANGCHAIN TRACING V2

LANGCHAIN PROJECT

Trace dashboard

LANGCHAIN API KEY

smith.langchain.com

TOPICS TO COMPLETE — check each as you finish the task

☐

Create free LangSmith account at smith.langchain.com

☐

Get LangSmith API key from Settings

☐

Set 3 environment variables in your notebook

☐

Run your Week 3 RAG chain — check traces appear in dashboard

☐

Open one trace — find the retrieved chunks, prompt, and response

☐

Ask a question that gives a wrong answer — read its trace to find why

☐

Check token count per step in the trace

WATCH BEFORE CODING

YouTube

CONCEPTS

LangSmith + LangChain Full AI Agent Tracing and Debugging (Feb 2026)

YouTube

CONCEPTS

Getting Started with LangSmith Tracing (Jun 2025)

**READ MORE — TOPIC REFERENCES (verified links)**

GFG

[GFG — Debugging and Testing LLMs in LangSmith](#)

GFG

[GFG — Tracing with LangChain](#)**COMMON ERRORS + QUICK FIXES****Traces not appearing in dashboard** → Check LANGCHAIN\_TRACING\_V2 is "true" (string) not True (boolean)**AuthenticationError: invalid API key** → Copy the full LangSmith key — it starts with "ls\_\_"**Project not visible in dashboard** → Create the project first at [smith.langchain.com/o/.../projects](https://smith.langchain.com/o/.../projects)**Traces show but steps missing** → Some LangChain community components do not auto-trace — use `verbose=True`**Step 1** Set up LangSmith — 3 environment variablesFree account at [smith.langchain.com](https://smith.langchain.com). Get API key from Settings > API Keys.

```
import os

# Get free API key from smith.langchain.com -> Settings -> API Keys
os.environ["LANGCHAIN_TRACING_V2"] = "true"
os.environ["LANGCHAIN_API_KEY"] = "your-langsmith-api-key"
os.environ["LANGCHAIN_PROJECT"] = "week4-rag-app"

# These 3 lines are all you need – no other code changes
# Every LangChain call after this is automatically traced
print("LangSmith tracing enabled")
print("Dashboard: https://smith.langchain.com")
```

**Week 3 connection:** That is it. No middleware, no wrappers, no code changes. Every LLM call you make after setting these variables gets logged to your LangSmith dashboard automatically.

**Step 2** Run your RAG chain and check the dashboardNew cell. Run a few queries then open [smith.langchain.com](https://smith.langchain.com) to see the traces.

```
# Load your Week 3 RAG chain (reuse your code)
# Then run these queries
test_queries = [
    "What is the main topic of this document?",
    "your weak question from Day 1",
    "a question with a known correct answer",
]

for q in test_queries:
    result = rag_chain.invoke({"input": q})
    print(f"Q: {q}")
    print(f"A: {result['answer'][:150]}\n")

print("\nCheck smith.langchain.com for traces")
```

**Note:** Open [smith.langchain.com](https://smith.langchain.com) now. You should see 3 traces under your project name. Click one — you will see the full prompt, retrieved chunks, and response all in one view.

**Step 3** Read a trace — find why a wrong answer happened

In the LangSmith dashboard. Click your weakest question trace.

```
# In the LangSmith dashboard:
# 1. Click on the trace for your weak question
# 2. Open the "Retriever" step – see which chunks were fetched
# 3. Open the "ChatGoogleGenerativeAI" step – see the full prompt
# 4. Check: were the right chunks retrieved?
#   If NO -> fix with hybrid search / reranking (Week 3 Day 4)
#   If YES but answer is wrong -> fix the system prompt

# Run this to see token breakdown in your notebook too
result = rag_chain.invoke({"input": "your weak question"})
print("Answer:", result["answer"])
print("Sources:", [d.metadata.get("page") for d in result["context"]])
print("\nCheck LangSmith trace for full token breakdown")
```

**Important:** This is the most valuable skill in Week 4 — using traces to diagnose exactly what went wrong. You will use LangSmith for the rest of your career building LLM apps.

### End of Day 2 — mark each one when done

- ☐ LangSmith account created at smith.langchain.com
- ☐ Three environment variables set correctly
- ☐ RAG chain runs and traces appear in dashboard
- ☐ Opened one trace and found the retrieved chunks section
- ☐ Identified what caused a wrong answer by reading the trace
- ☐ Notebook saved as week4\_day2\_langsmith.ipynb

**NEXT:** Day 3: Add error handling and guardrails to your Streamlit app — what to do when the document doesn't have the answer, when the API fails, and how to block harmful queries.

MY NOTES — write your key takeaway before moving on

Takeaway: \_\_\_\_\_

Surprised by: \_\_\_\_\_

Question: \_\_\_\_\_

## Day 3 Error handling + guardrails | Make the app reliable

~1.5 hrs

**ORG CONNECTION:** Error handling and guardrails are what your org's security review will check before approving any AI app deployment

LLM layer

### Theory — Guardrails and fallback patterns for RAG apps

#### TOPICS IN THIS SECTION:

Input guardrails

Hallucination

try/except

Fallback response

Error handling

API timeout

#### TOPICS TO COMPLETE — check each as you understand it

- ☐ Three layers of guardrails: input, retrieval, output
- ☐ Why checking retrieval score before generating prevents hallucination
- ☐ How to detect when no relevant chunks were found
- ☐ How try/except wrapping prevents app crashes on API errors
- ☐ How to set a timeout on LLM calls for production reliability

A production RAG app needs three layers of protection. Input guardrails check the query before it reaches the LLM — reject questions unrelated to the document or containing harmful content. Retrieval guardrails check that relevant chunks were actually found before generating an answer. Output guardrails verify the answer references the document and does not hallucinate. Each layer prevents a different class of failure.

**FOCUS ON** Check if retrieved chunks are actually relevant before generating | Wrap every API call in try/except

**SKIP FOR NOW** Constitutional AI, Guardrails AI framework — overkill for first org project

#### RESOURCES

- ☐ **YouTube** [Production RAG with LangChain full course \(May 2026\)](#)
- ☐ **Blog** [RAG with LangChain — guardrails section](#)
- ☐ **Blog** [LangChain Integrations — error handling](#)

#### READ MORE — TOPIC REFERENCES

**GeeksForGeeks**

[RAG with LangChain — reliability patterns](#)

**GeeksForGeeks**

[LangChain Integrations — error handling](#)

**GeeksForGeeks**

[Complete LangChain Ecosystem](#)

— Take a 10 min break before starting practice —

**GenAI layer**

### Practice — Add guardrails and error handling to your Streamlit app

#### TOPICS IN THIS SECTION:

**Input validation**

**try/except**

**Empty query check**

**Fallback message**

**st.error()**

#### TOPICS TO COMPLETE — check each as you finish the task

- ☐ Add input validation — check query is not empty or too short
- ☐ Add relevance check — count retrieved chunks before generating
- ☐ Add fallback message when no relevant chunks found
- ☐ Wrap `rag_chain.invoke()` in try/except in Streamlit app
- ☐ Display `st.error()` for API failures instead of crashing
- ☐ Add a minimum query length check (at least 5 characters)

#### WATCH BEFORE CODING

**YouTube**

CONCEPTS

[Production RAG with LangChain full course \(May 2026\)](#)

#### READ MORE — TOPIC REFERENCES (verified links)

**GFG**

[GFG — RAG with LangChain](#)

**GFG**

[GFG — Streamlit beginner guide](#)

#### COMMON ERRORS + QUICK FIXES

**Guardrail blocks legitimate queries** → `Off_topic_signals` list is too broad — keep it specific to your

**try/except swallows all errors silently** → Always print or log the error — use `st.error()` not `pass`

**Fallback message appears even with correct answers** → Check `result["context"]` not `result["answer"]` — context might b

**st.warning does not stop the chain** → Add `return` or `st.stop()` after `st.warning()` to prevent executio

#### Step 1 Add input guardrail — validate query before sending to LLM

Open your Streamlit app file. Add this before `rag_chain.invoke()`.



```
def validate_query(query: str) -> tuple[bool, str]:
    """Return (is_valid, error_message)"""
    if not query or len(query.strip()) < 5:
        return False, "Query too short – please ask a complete question."
    if len(query) > 500:
        return False, "Query too long – please keep it under 500 characters."
    # Block obviously off-topic queries
    off_topic_signals = ["weather", "recipe", "joke", "who is your creator"]
    if any(signal in query.lower() for signal in off_topic_signals):
        return False, "That question seems unrelated to the document. Please ask about t
    return True, ""

# In your Streamlit chat handler:
# if q := st.chat_input("Ask about your document..."):
#     is_valid, error_msg = validate_query(q)
#     if not is_valid:
#         st.warning(error_msg)
#     else:
#         # proceed with RAG chain
```

**Note:** Keep the `off_topic_signals` list short and document-specific. Add signals that make sense for your org's use case — not a generic filter.

## Step 2 Add retrieval fallback and API error handling

New cell. Wrap your RAG chain call with proper error handling.

```
def safe_rag_query(rag_chain, question: str) -> dict:
    """Run RAG with fallback for API errors and missing content."""
    try:
        result = rag_chain.invoke({"input": question})

        # Check if any chunks were retrieved
        if not result.get("context"):
            return {
                "answer": "I could not find relevant information in the document for thi
                "sources": [],
                "error": None
            }

        pages = sorted(set(
            d.metadata.get("page", "?") for d in result["context"]
        ))
        return {
            "answer": result["answer"],
            "sources": pages,
            "error": None
        }

    except Exception as e:
        return {
            "answer": None,
            "sources": [],
            "error": f"API error: {str(e)[:100]}. Please try again."
        }

# In Streamlit app:
# result = safe_rag_query(rag_chain, question)
# if result["error"]:
#     st.error(result["error"])
# else:
#     st.chat_message("assistant").write(result["answer"])
```

**Week 3 connection:** This wrapper means your Streamlit app never crashes. Every error is caught and shown to the user clearly. This is the difference between a demo and a production app.

#### End of Day 3 — mark each one when done

- ☐ validate\_query() function added and tested with short and off-topic queries
- ☐ safe\_rag\_query() wrapper added — handles API errors without crashing
- ☐ Streamlit app shows st.error() instead of crash on API failure
- ☐ Fallback message appears when no chunks are found
- ☐ Tested by asking a question completely unrelated to the document
- ☐ Notebook/app saved as week4\_day3\_guardrails.py

**NEXT:** Day 4: Polish the Streamlit app — loading spinners, expandable source sections, token usage display, and a clear chat button. Turns your working prototype into something you'd be proud to show.

MY NOTES — write your key takeaway before moving on

Takeaway:

Surprised by:

Question:

Day 4Streamlit polish | From working to impressive

~1.5 hrs

ORG CONNECTION: A polished UI with session metrics is what turns a proof-of-concept into something your org will actually fund

LLM layer

Theory — What makes an AI app feel production-ready

TOPICS IN THIS SECTION:

st.spinner()

st.expander()

st.sidebar

st.session\_state

st.rerun()

UX patterns

TOPICS TO COMPLETE — check each as you understand it

☐ st.spinner() — shows loading message while RAG chain runs

☐ st.expander() — collapses source documents to keep UI clean

☐ Showing token count per query in the sidebar

☐ st.rerun() — how to force a rerender after state change

☐ st.set\_page\_config() — title, icon, layout for professional look

☐ Sidebar metrics — total queries, total tokens used so far

A working app and a polished app are different in five ways: loading feedback (spinner while processing), structured output (expandable sections), transparency (token usage and source display), control (clear history button), and error clarity (descriptive messages not stack traces). Streamlit provides all of these as built-in components — they take 10 minutes to add but make the app feel 10x more professional.

FOCUS ON st.spinner + st.expander are the two highest-impact polish additions | Sidebar metrics show stakeholders the app is being used

SKIP FOR NOW Custom CSS, Streamlit themes, multi-page apps — future enhancement

RESOURCES

☐

YouTube

 RAG Onboarding Chatbot Streamlit+ChromaDB (Sep 2025)

☐

Blog

 Streamlit beginner guide — GFG

☐

Blog

 Deploy AI model using Streamlit — GFG

☐

Course

 Streamlit official API reference

READ MORE — TOPIC REFERENCES

GeeksForGeeks

Streamlit beginner guide — all UI components

GeeksForGeeks

Deploy AI model with Streamlit

— Take a 10 min break before starting practice —

GenAI layer

Practice — Add spinner, expander, token display, and sidebar metrics

TOPICS IN THIS SECTION:

st.spinner()

st.expander()

token\_count

st.metric()

st.set\_page\_config

**TOPICS TO COMPLETE — check each as you finish the task**

- ☐ Add `st.set_page_config(page_title, page_icon, layout)` at top
- ☐ Wrap RAG chain call in with `st.spinner("Searching document...")`
- ☐ Show sources in `st.expander("Sources", expanded=False)`
- ☐ Display query token count after each answer
- ☐ Add sidebar `st.metric()` for total queries and total tokens
- ☐ Add `st.divider()` between messages for visual separation
- ☐ Test the complete polished app end to end

**WATCH BEFORE CODING**

YouTube

CONCEPTS

[RAG Onboarding Chatbot Streamlit+LangChain+ChromaDB \(Sep 2025\)](#)**READ MORE — TOPIC REFERENCES (verified links)**

GFG

[Streamlit API reference — official](#)

GFG

[GFG — Streamlit beginner guide](#)**COMMON ERRORS + QUICK FIXES****st.spinner does not show** → The code inside with `st.spinner()` must actually take time — in**Expander opens by default** → Set `expanded=False` in `st.expander()` to keep sources collapsed**Metrics reset on every query** → Metrics must be in `st.session_state` — not regular variables wh**Page config error: already set** → `st.set_page_config()` must be the FIRST Streamlit call — before**Step 1** Add professional page config and spinner

Open your Streamlit app. Add these at the top and around the RAG call.

```
# At the very top of your streamlit app:
st.set_page_config(
    page_title="Document Q&A",
    page_icon="■",
    layout="centered"
)

# Replace your existing rag_chain.invoke() call with:
if q := st.chat_input("Ask about your document..."):
    st.session_state.messages.append({"role": "user", "content": q})
    st.chat_message("user").write(q)

    with st.spinner("■ Searching document..."):
        result = safe_rag_query(rag_chain, q) # your Day 3 wrapper

    if result["error"]:
        st.error(result["error"])
    else:
        # Show answer
        st.chat_message("assistant").write(result["answer"])

        # Show sources in collapsible section
        with st.expander(f"■ Sources: pages {result['sources']}"):
            for doc in result.get("context_docs", []):
                st.caption(f"Page {doc.metadata.get('page', '?')}:")
                st.text(doc.page_content[:300])
```

**Week 3 connection:** The expander keeps the chat clean. Users see the answer immediately and can expand to verify the source if they want. This is the right default UX for a document Q&A app.

## Step 2 Add sidebar metrics and token tracking

Add to your sidebar block.

```
# In the sidebar, after the file uploader:
if "total_queries" not in st.session_state:
    st.session_state.total_queries = 0
    st.session_state.total_tokens = 0

st.sidebar.divider()
st.sidebar.subheader("Session Stats")
st.sidebar.metric("Queries asked", st.session_state.total_queries)
st.sidebar.metric("Total tokens used", st.session_state.total_tokens)

# After each successful query, increment:
# st.session_state.total_queries += 1
# st.session_state.total_tokens += estimated_tokens

# Estimate tokens: len(question.split()) * 1.3 + len(answer.split()) * 1.3
estimated = int((len(q.split()) + len(result["answer"].split()))) * 1.3
st.session_state.total_tokens += estimated
st.session_state.total_queries += 1
```

**Note:** The session metrics show stakeholders how actively the app is being used during a demo. Total tokens gives a rough idea of API cost per session.

### End of Day 4 — mark each one when done

- ☐ st.set\_page\_config with title and icon added
- ☐ st.spinner wraps the RAG chain call
- ☐ Sources shown in st.expander (collapsed by default)
- ☐ Sidebar shows total queries and estimated tokens
- ☐ Clear chat button resets messages and metrics
- ☐ App tested end to end — upload PDF, ask 5 questions, verify all UI elements work
- ☐ App saved as streamlit\_rag\_app\_final.py

**NEXT:** Day 5: Prepare your 5-minute team demo. Practice your walkthrough. Show the polished Streamlit app running on real org documents. This is your entry into the org AI project.

### MY NOTES — write your key takeaway before moving on

Takeaway: \_\_\_\_\_

Surprised by: \_\_\_\_\_

Question: \_\_\_\_\_

## Day 5 Team demo + project presentation | 4 weeks of work in 5 minutes

~2 hrs

**ORG CONNECTION:** This demo is your entry into the org AI project — not just a learning exercise but a real deliverable that shows your value

LLM layer

### Theory — How to present an AI project to non-technical stakeholders

TOPICS IN THIS SECTION:

Demo structure

Business problem

4-week summary

Stakeholder comms

Next steps

What you built

TOPICS TO COMPLETE — check each as you understand it

☐ Demo structure: problem → live demo → architecture → next steps

☐ How to explain RAG to a non-technical manager in 30 seconds

☐ Which questions to anticipate and prepare answers for

☐ How to show LangSmith traces during the demo to show reliability

☐ What to say when the demo fails live (always have a backup)

Most AI demos fail because developers explain how the technology works instead of showing what the business problem it solves. A good 5-minute demo has four parts: the problem (what takes time today), the live demonstration (upload document, ask real questions), the architecture (one sentence each for embedding, retrieval, generation), and next steps (what could be improved). Never start with how the model works.

FOCUS ON    Start with the business problem, not the technology | Live demo on real org documents makes the biggest impression

SKIP FOR NOW    Deployment to cloud, authentication, multi-user support — next phase after approval

RESOURCES

☐

YouTube

 Production RAG with LangChain full course (May 2026)

☐

Blog

 RAG with LangChain — GFG complete guide

☐

Blog

 Complete LangChain Ecosystem — GFG

READ MORE — TOPIC REFERENCES

GeeksForGeeks

RAG with LangChain — complete reference

GeeksForGeeks

Complete LangChain Ecosystem

GeeksForGeeks

Introduction to LangChain — overview

— Take a 10 min break before starting practice —

GenAI layer    Practice — Prepare and rehearse the 5-minute team demo

TOPICS IN THIS SECTION:

Demo prep

LangSmith trace

Backup plan

Real org doc

5 questions ready

TOPICS TO COMPLETE — check each as you finish the task

☐ Prepare 5 real questions to ask live during the demo

☐ Run the app on the org document ahead of time — no surprises

☐ Prepare a 30-second explanation of what RAG does in plain English

☐ Screenshot the LangSmith dashboard to show traceability

☐ Prepare backup: screenshots of working app in case of live failure

☐ Write a one-paragraph summary of what you built to share with the team

☐ Rehearse the demo twice before the actual presentation

WATCH BEFORE CODING

YouTube

CONCEPTS

RAG Onboarding Chatbot Streamlit+ChromaDB (Sep 2025)

**READ MORE — TOPIC REFERENCES (verified links)**

GFG

[GFG — RAG with LangChain](#)

GFG

[GFG — Complete LangChain Ecosystem](#)**COMMON ERRORS + QUICK FIXES****Demo fails live due to API rate limit** → Pre-run all 5 questions before the demo — Streamlit caches ses**Manager asks "can it do X?"** → Answer: "Not yet in this version — that's on the roadmap for P**LangSmith traces not visible during demo** → Open the dashboard in a separate browser tab before starting —**Streamlit app crashes during demo** → Have screenshots backup in a PowerPoint — show those calmly an**Step 1** Write your 5-minute demo script

Open a text file or Google Doc. Write out exactly what you will say.

## # 5-MINUTE DEMO SCRIPT TEMPLATE

# MINUTE 1 — THE PROBLEM (no technology yet)

# "Right now when a new employee asks about our [HR/policy/product] document,  
# they either read through 50 pages manually or ask a colleague.

# Both take time. I built a system that answers questions from our documents instantly."

# MINUTE 2 — LIVE DEMO (do this live)

# Upload org document to Streamlit app

# Ask 5 prepared questions

# Show source citations appearing

# Ask one question NOT in the document to show the fallback

# MINUTE 3 — HOW IT WORKS (30 seconds max)

# "The document is split into small pieces and converted into numbers

# (embeddings) that capture meaning. When you ask a question, the system

# finds the most relevant pieces and gives them to the AI to generate an answer.

# Only the document — nothing from the internet."

# MINUTE 4 — SHOW RELIABILITY

# Open LangSmith dashboard

# "Every query is logged — we can see exactly what it retrieved and why."

# MINUTE 5 — NEXT STEPS

# "This runs on any PDF. Next: user authentication, more documents,

# deployment to the internal server. What documents would be most useful?"

**Week 3 connection:** The most important sentence is in Minute 1. If they do not understand the problem you are solving, the rest does not matter. Practice that sentence until it takes 20 seconds exactly.**Step 2** Run the full demo dry-run

Do the entire demo exactly as you would in front of your team. Time it.

```
# Pre-demo checklist:
print("""
PRE-DEMO CHECKLIST:
■ Org document loaded in Streamlit (already embedded – no waiting)
■ 5 test questions written and tested ahead of time
■ LangSmith dashboard open in browser tab
■ Backup screenshots ready in a folder
■ API key in environment (not typed live)
■ streamlit run streamlit_rag_app_final.py already running
■ Browser font size set to readable (Ctrl+= to zoom in)
■ Chat history cleared for a clean start

Your 4-week output:
  Week 1: Gemini API, embeddings, chatbot with memory
  Week 2: LangChain chains, memory, agents, document loaders
  Week 3: Full RAG app with hybrid search and reranking
  Week 4: Tuned, debugged, polished, production-ready RAG app

Running this entire session: python3 -c \"import subprocess; subprocess.run(['streamlit'
""")
```

**Note:** Run the dry-run at least twice. The first time shows you what will go wrong. The second time you fix it. The third time — in front of your team — goes smoothly.

#### End of Day 5 — mark each one when done

- ☐ Demo script written — problem, live demo, architecture, next steps
- ☐ 5 questions prepared and pre-tested on the org document
- ☐ LangSmith dashboard has traces from test runs
- ☐ Pre-demo checklist completed
- ☐ Dry-run completed twice — timing under 5 minutes
- ☐ Backup screenshots ready
- ☐ Demo presented to team or manager

**NEXT:** After Week 4: LangGraph for multi-agent workflows, LangSmith evaluators, cloud deployment, and RAG Fusion. Follow Balaji Chippada's Phase 5+ roadmap (Telugu) for the full Agentic AI Engineer path.

#### MY NOTES — write your key takeaway before moving on

Takeaway: \_\_\_\_\_

Surprised by: \_\_\_\_\_

Question: \_\_\_\_\_



# Week 4 Resources — All Verified Links

Every link below has been verified working. No GitHub links. Click directly from this page.

## YouTube — Week 4 specific (all verified 2025-2026)

- ☐ [LangSmith + LangChain Full AI Agent Tracing and Debugging \(Feb 2026\)](#)
- ☐ [Trace and Debug LangChain Agents with LangSmith \(Apr 2026\)](#)
- ☐ [LangSmith Tracing Tutorial step by step \(Jan 2026\)](#)
- ☐ [Getting Started with LangSmith Tracing \(Jun 2025\)](#)
- ☐ [Production RAG with LangChain full course \(May 2026\)](#)
- ☐ [RAG Onboarding Chatbot Streamlit+ChromaDB \(Sep 2025\)](#)

## GFG — reading references (all verified)

- ☐ [Debugging and Testing LLMs in LangSmith](#)
- ☐ [Tracing with LangChain — full setup guide](#)
- ☐ [Introduction to LangSmith](#)
- ☐ [RAG with LangChain — complete guide](#)
- ☐ [Complete LangChain Ecosystem](#)
- ☐ [RAGAS — RAG evaluation framework](#)
- ☐ [Streamlit beginner guide](#)

## Free courses and official docs

- ☐ [Streamlit official API reference](#)
- ☐ [LangSmith official docs](#)
- ☐ [DeepLearning.AI — LangChain Chat with Your Data \(free\)](#)